

s3.06 — Durable distributed event log — ordered partitions and replay

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Append, replication, and retry boundaries — IC3 baseline](#)
- [Consumer progress, ordering, and retention — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` changes the architecture only when a stronger requirement earns the cost.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: [ranked bank](#).
 - Prior supplemental bank and preparation addition.
 - This is a proposed design, not a claim about OCI internals or a reported accepted answer.
- Why this is hard
 - Naive: append events to a single file and return success when `write()` returns.
 - First break: the leader dies before replicated durability, or a consumer repeats an external effect after processing but before committing its offset.
 - Second break: adding partitions moves a key, so records for one key now sit in two partitions with no order between them.
 - Minimum primitives
 - Ordered append-only partitions.
 - Replicated acknowledgment policy.
 - Producer retry identity.
 - Consumer assignment and contiguous progress tracking.
- 🗣️ "The log orders durable records; the consumer owns what those records do outside the log."
- Opening script
 - "I will design an ordered partitioned log with replay and explicit acknowledgment semantics."
 - "I will clarify the ordering key, retention, and the failure set that an acknowledged event must survive."
 - "Each partition will have one leader and replicated followers."
 - "The deep dive is retrying a timed-out append and committing consumer progress without skipping unfinished records."
 - "I will close with retention, rebalances, and the boundary of exactly-once processing."

Phase	Clock	Deliverable
Intro	00:00–05:00	Establish role context and confirm the prompt
Requirements	05:00–11:00	Scope and one correctness invariant
Capacity and API	11:00–17:00	A number that forces a choice; retry contract
Architecture	17:00–30:00	Draw ownership and trace one request
Deep dive	30:00–45:00	Concurrency proof and a crash boundary
Operations and alternatives	45:00–50:00	Degradation, recovery, and flip condition
Wrap-up	50:00–54:00	Four-sentence recap and unresolved assumption
Interviewer questions	54:00–60:00	Team ownership and production priorities

- Clock note
 - The design slice is 45 minutes within the 60-minute round.
 - If the interviewer gives 45 minutes total, use a 2-minute intro and compress the remaining phases.
 - The interviewer's clock wins.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Ordering scope?	Per partition; same key stays in one partition	No global total order
Failure tolerance?	One broker failure with replication factor 3	Acknowledgment and election policy matter
Consumer behavior?	Independent groups with replay	Store progress per group and partition
Retention?	7 days time retention baseline	Consumers can fall behind and lose replay access
Exactly once where?	Kafka-like transactions only inside supported log operations	External effects need destination cooperation

- Functional scope
 - Append records with keys and producer identity.
 - Read by partition offset.
 - Replicate partitions and elect leaders.
 - Track consumer-group assignment and progress.
 - Retain segments and support replay within retention.
- Targets below are interview assumptions, not product SLAs.

Requirement	Target	Enforced by
Append latency	p99 < 20 ms regional assumption	Batching and replicated acknowledgments
Availability	99.9% target under 1 broker loss	3 replicas and clean leader election
Durability	0 acknowledged loss under stated surviving-replica assumptions	acks=all with minimum ISR 2; storage contract explicit
Retention	7 days baseline	Segment retention policy and capacity admission
Record size	Maximum 1 MB	Producer/broker validation
Consumer recovery	Target < 30 s after session failure	Group timeout and reassignment

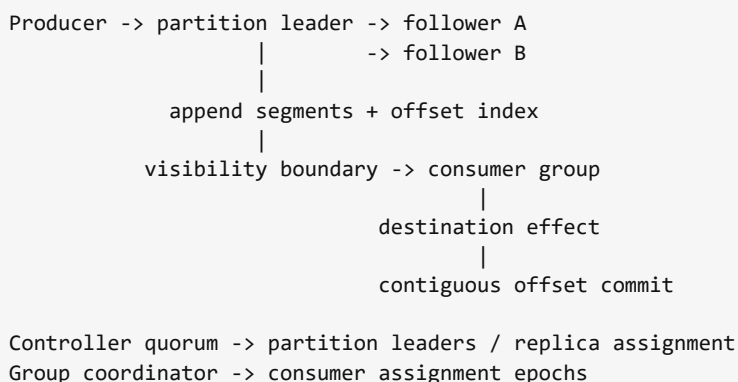
- Invariants
 - Each partition exposes records in increasing offset order.

- A producer retry identity cannot append the same accepted sequence twice within its supported epoch/session contract.
- Consumers only observe the visibility boundary promised by the read mode.
- Committed consumer progress never skips unfinished lower offsets.
- Retention loss is reported explicitly rather than silently resetting a consumer.
- Non-goals
 - Global ordering across partitions.
 - Exactly-once arbitrary external side effects.
 - Unlimited replay after retention expires.

Capacity and consequence — IC3 baseline

- All workload values below are sizing assumptions.
- Ingest
 - Assume $500,000 \text{ records/s} \times 1 \text{ KB} = 500 \text{ MB/s}$ logical ingress.
 - Decision: batch appends and distribute partition leaders across brokers.
- Retention
 - $500 \text{ MB/s} \times 86,400 \times 7 = 302.4 \text{ TB}$ logical seven-day data.
 - Replication factor 3 gives 907.2 TB before indexes and filesystem overhead.
 - Decision: compression and tiered storage are cost decisions, not optional hand-waving.
- Replica traffic
 - Two follower copies add about 1 GB/s aggregate leader-to-follower traffic before protocol overhead.
 - Decision: size network and disk jointly under one-broker loss and replica catch-up.
- Consumers
 - Four full-read groups add roughly 2 GB/s aggregate uncompressed payload delivery.
 - Decision: model egress separately from ingress and isolate slow consumers.
- Partitions
 - If a benchmark supports 10 MB/s logical writes per partition for this latency target, ingress alone needs at least 50 partitions.
 - Decision: choose extra measured headroom and consumer parallelism without presenting that benchmark as a product constant.

Architecture and flow — IC3 baseline



- Partition leader
 - Buys ordered append and batching.
 - Pays a throughput ceiling for one hot partition.
- Replicas
 - Buy survival of the declared broker failure set.
 - Pay network, disk, and catch-up IO.

- Controller and group coordinator
 - Own topology and assignment epochs.
 - Pay coordination and rebalance cost.
 - Consumer
 - Owns effect completion and contiguous progress.
 - Pays destination idempotency and replay handling.
- Map the stable key to a partition using the declared partitioning version.
- Send producer ID, epoch, sequence, and record batch.
- Leader appends and replicates according to the acknowledgment policy.
- Return offsets after the required replication boundary is met.
- Assigned consumer reads visible records and commits destination effects.
- Commit only the next offset after the contiguous completed prefix.

API and data model — IC3 baseline

- Proposed API contracts.
 - Authenticate the tenant from credentials before looking up any resource.
 - Retain idempotency records for the documented retry horizon.

Surface	Contract	Idempotency and conflicts
Append(topic, partition, producer, epoch, sequence, batch)	Returns base/last offsets after requested acknowledgment	Retry identity deduplicates within producer protocol; invalid sequence or stale epoch rejected
Fetch(topic, partition, offset, maxBytes, isolation)	Returns records and visibility metadata	Offset below retained start returns explicit out-of-range
JoinGroup(group, member)	Assignment and generation	Stale member generation cannot commit group progress
CommitOffset(group, partition, nextOffset, generation)	Persists next unread position	Reject stale assignment; client must supply contiguous progress
Describe(topic/partition)	Leader, retained start, end and configuration	Metadata can become stale; retry after leader change

State	Key	Fields and invariant
Segment	(topic, partition, baseOffset)	Framed records with length and checksum
Sparse index	(partition, baseOffset, relativeOffset)	Byte position; rebuildable from verified frames
Producer state	(producerId, epoch, partition)	Last accepted sequence and retry result
Replica state	(partition, brokerId)	Log end and in-sync status
Group progress	(groupId, partition)	Committed next offset and assignment generation
Transaction state	(transactionId, producerEpoch)	Supported log transaction status and visibility markers

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Records live in log segments on broker disks; only topic and partition metadata, replica membership, consumer group progress, and retention policy are relational.

```

CREATE TABLE topics (
  tenant_id text NOT NULL, topic_id text NOT NULL, name text NOT NULL,
  partition_count integer NOT NULL, replication_factor integer NOT NULL,
  min_in_sync_replicas integer NOT NULL,
  retention_ms bigint NOT NULL, retention_bytes bigint,           -- NULL = no size bound
  cleanup_policy text NOT NULL DEFAULT 'delete',
  tombstone_retention_ms bigint,                                   -- compaction only
  version bigint NOT NULL,                                         -- PATCH precondition
  PRIMARY KEY (tenant_id, topic_id),
  UNIQUE (tenant_id, name),
  CHECK (partition_count >= 1),
  CHECK (replication_factor >= min_in_sync_replicas AND min_in_sync_replicas >= 1),
  CHECK (retention_ms > 0),
  CHECK (cleanup_policy IN ('delete', 'compact', 'compact,delete'))
);
CREATE TABLE partitions (
  tenant_id text NOT NULL, topic_id text NOT NULL, partition integer NOT NULL,
  leader_broker_id text,                                           -- NULL while an election is pending
  leader_epoch bigint NOT NULL,                                     -- fences stale leaders on append and
  replica fetch
  retained_start_offset bigint NOT NULL DEFAULT 0,                 -- advances on segment deletion
  high_watermark bigint NOT NULL DEFAULT 0,                       -- controller's reported view, not the
  leader's log
  PRIMARY KEY (tenant_id, topic_id, partition),
  FOREIGN KEY (tenant_id, topic_id) REFERENCES topics,
  CHECK (partition >= 0),
  CHECK (retained_start_offset <= high_watermark)
);
CREATE INDEX partitions_by_leader ON partitions (leader_broker_id, tenant_id, topic_id, partition);
CREATE TABLE partition_replicas (
  tenant_id text NOT NULL, topic_id text NOT NULL, partition integer NOT NULL,
  broker_id text NOT NULL,
  in_sync boolean NOT NULL DEFAULT false,
  log_end_offset bigint NOT NULL DEFAULT 0,                       -- last reported by the broker; may lag
  PRIMARY KEY (tenant_id, topic_id, partition, broker_id),
  FOREIGN KEY (tenant_id, topic_id, partition) REFERENCES partitions,
  CHECK (log_end_offset >= 0)
);
CREATE INDEX replicas_by_broker ON partition_replicas (broker_id) WHERE in_sync;
CREATE TABLE consumer_groups (
  tenant_id text NOT NULL, group_id text NOT NULL,
  generation bigint NOT NULL DEFAULT 0, protocol text NOT NULL,
  state text NOT NULL, coordinator_broker_id text,
  PRIMARY KEY (tenant_id, group_id),
  CHECK (state IN ('EMPTY', 'REBALANCING', 'STABLE', 'DEAD'))
);
CREATE TABLE group_offsets (
  tenant_id text NOT NULL, group_id text NOT NULL,
  topic_id text NOT NULL, partition integer NOT NULL,
  next_offset bigint NOT NULL,                                     -- next record to process, not last
  fetched
  group_generation bigint NOT NULL,                                 -- commit rejected when below the group's
  generation
  committed_at timestampz NOT NULL,
  PRIMARY KEY (tenant_id, group_id, topic_id, partition),
  FOREIGN KEY (tenant_id, group_id) REFERENCES consumer_groups,
  FOREIGN KEY (tenant_id, topic_id, partition) REFERENCES partitions,
  CHECK (next_offset >= 0)
);
CREATE INDEX offsets_by_partition ON group_offsets (tenant_id, topic_id, partition, next_offset);

```

- What the DDL does not capture
 - Segments, sparse indexes, and record bytes live in broker log files; producer state is a per-partition snapshot rebuilt from verified frames, and transaction state lives in the coordinator's internal compacted log.

- `high_watermark` and `log_end_offset` are reported views for lag and retention alarms; the partition leader's log is the source of truth for what is committed.
- Deviation from the companion and the state table
 - `tenant_id` is added to every key because quotas and retention budgets are enforced per tenant.
 - `Replica state` becomes the controller's ISR view in `partition_replicas`, standing in for the controller quorum's metadata log.

Append, replication, and retry boundaries — IC3 baseline

- Append storage
 - Frame each record or batch with length and checksum.
 - Track the active file position incrementally.
 - A sparse offset index maps logical offsets to nearby byte positions.
 - Recovery scans verified frames and truncates an incomplete tail.
 - Indexes are rebuildable
 - Committed data is not reconstructed from an index alone.
- Kafka-like acknowledgment policy
 - `acks=all` waits for the current in-sync replica set according to the broker protocol.
 - `min.insync.replicas=2` is an admission threshold for that acknowledgment mode.
 - It is not the same algorithm as a Raft majority commit.
 - With replication factor 3, losing one replica can leave two eligible replicas and continued writes.
 - Disable unclean leader election when preserving acknowledged records takes priority over availability.
- Storage defeater
 - Replication into operating-system page caches is not the same as an `fsync` on every append.
 - State the chosen broker/storage durability contract and correlated power-loss assumptions.
 - A guarantee against one broker loss does not imply survival of every replica losing volatile state together.
- Producer retries
 - An append timeout is ambiguous.
 - Retry with the same producer epoch and sequence under the idempotent producer protocol.
 - A new producer session or a new business request is not automatically deduplicated against all history.
 - Keep a business event ID if duplicate intent can originate outside that protocol.
- Visibility
 - The high watermark bounds the replicated prefix visible to ordinary consumers.
 - Transactional `read_committed` may stop earlier at the last stable offset while transactions remain open.

Consumer progress, ordering, and retention — IC3 baseline

- Group ownership
 - One logical member owns a partition within a group assignment generation.
 - A paused former member can still have in-flight external requests.
 - Coordinator fencing protects group commits, not arbitrary destination effects.
- Effect then progress
 - Commit a destination effect before advancing the consumer offset.
 - A crash between them causes replay.
 - Deduplicate by event identity with the destination effect in one transaction.
 - Kafka transactions can coordinate supported Kafka records and offsets, not an unrelated database automatically.
- Parallel handling
 - Track completed offsets and advance only through the contiguous prefix.
 - If offsets 10 and 12 finished but 11 did not, commit next offset 11.
 - Committing 13 would skip unfinished record 11 after restart.

- Partition growth
 - Increasing partition count can remap a key under modulo hashing.
 - Old and new partitions do not share a total order.
 - If key order matters, version routing and drain/fence old producers before switching that key.
- Retention and compaction
 - Time retention deletes eligible old segments regardless of a slow consumer's progress.
 - Expose earliest retained offset and alarm before consumer lag reaches the boundary.
 - Log compaction retains a latest-value view eventually, subject to its cleaner and tombstone policies.
 - Compaction is not immediate deletion or an exact per-key TTL.
- Poison records
 - Bound retries and use an explicit quarantine policy.
 - Preserve the original topic, partition, offset, and failure reason.
 - Skipping a record changes the application contract and must be visible.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Leader dies after append reply	One partition	Leader heartbeat	Clean eligible replica takes over under stated durability assumptions	Retry ambiguous records with producer identity
ISR drops below minimum	Affected partitions	ISR count and append errors	Reject durable writes	Recover replicas before reopening
Consumer dies after effect	One group partition	Session timeout	New member replays	Destination deduplication returns prior result
Slow consumer crosses retention	One group	Lag versus earliest retained offset	Explicit out-of-range	Restore from archive or perform declared resnapshot
Hot partition	One key range	Per-partition latency and bytes	Backpressure that partition	Change key distribution only with an ordering migration
Controller quorum unavailable	Leader elections and topic changes	Controller health	Existing partition leaders keep serving appends and fetches	Restore controller quorum; pending elections complete
Group coordinator unavailable	Commits and rebalances for its groups	Commit errors	Consumers keep processing on the last assignment; commits retry	Coordinator moves to another broker; replay from last commit
Broker disk full	Partitions on that broker	Disk usage and append errors	Follower drops from ISR; leader rejects writes if minimum ISR is unmet	Expand or trim retention, then catch up
Poison record	One partition of one group	Retry count on the record	Quarantine with topic, partition, offset, and reason	Fix the consumer; replay quarantined offsets

- Observe
 - Under-replicated partitions and ISR changes.
 - Append latency and replica catch-up IO.
 - Consumer lag in both offsets and time.
 - Oldest open transaction and retained-start movement.
- Security
 - Encrypt broker traffic and restrict retention-policy changes.
 - Audit deletion and retention changes as control-plane operations.
- Retry discipline
 - Producer retries reuse the same producer epoch and sequence, with backoff and jitter, within the delivery timeout.
 - A stale-epoch or out-of-sequence rejection is a corrective action: re-initialize the producer, do not retry blindly.

- Consumers retry a failed effect a bounded number of times, then quarantine the record and leave the offset uncommitted.
- An offset-commit rejection for a stale generation means rejoin and re-read the assignment, never a forced commit.
- Tenant isolation
 - Enforce tenant byte-rate and storage quotas.
 - Authorize topic access and consumer-group operations independently.
 - Place critical tenants' partitions in separate broker cells so one tenant's hot partition cannot fill a shared disk.
 - Budget retention per tenant so one tenant's backlog cannot push another's segments off disk.
 - A tenant's credentials receive 404 on another tenant's topics, not 403, to avoid existence leaks.
- Rollout and rollback safety
 - Preserve replica availability during rolling upgrades.
 - Version record schemas and test old consumers before adding required fields.
 - Bump the inter-broker protocol and log format only after every broker runs the new version, so a rollback never meets an unreadable segment.
 - Canary one broker, watch under-replicated partitions and append latency, then continue.
 - Reassign replicas gradually to avoid saturating replication bandwidth.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Replicated event log	Ordered retained partitions	Replay and independent consumers	Retention and partition operations	Baseline event backbone
Task queue	Work delivery with acknowledgments	Simple competing workers	Replay and ordering may be limited	Independent jobs rather than event history
Database outbox plus relay	Business state and event intent	Atomic source transaction	Relay lag and duplicate delivery	Events originate with a database mutation
Object-storage archive	Immutable batch history	Low retention cost	Higher retrieval latency	Long replay horizon beyond hot storage

- Stronger requirement
 - Kafka-to-Kafka exactly-once processing uses supported transactions and read_committed consumers.
 - External destinations still require their own atomic effect boundary.
- At 10× load
 - Add partition cells and tier cold segments to object storage.
 - Model replica recovery bandwidth before selecting broker count.
- Cost and blast radius
 - Separate critical topics from noisy bulk ingestion.
 - Enforce retention budgets so one tenant cannot exhaust the shared disk.
- Migration
 - Version producer routing and drain old key ownership before repartitioning ordered streams.
- Weakest assumption
 - The selected failure domains and storage contract match the stated durability guarantee.
- Over-engineering warning
 - Do not promise global ordering or exactly-once external effects by merely enabling a producer setting.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Scope	Define partition ordering and replay horizon	Negotiate destination transaction boundary	No unqualified exactly-once claim
Capacity	Calculate replication and retention bytes	Tiering and recovery bandwidth	Includes cost beyond ingress
Correctness	Explain ISR and producer identity	Transaction visibility and epoch migration	Does not confuse Kafka with Raft
Operations	Commit contiguous progress	Repartition without breaking key order	Recovery preserves semantics
Architecture	Leader, replicas, controller, coordinator, consumer, one trace	Cost of each split and static stability under controller loss	Every component has a mechanism and a price
Wrap	Recap with retention and hot partitions as limits	Names the storage-contract assumption as the weakest	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Does acks=all mean a majority vote?
 - **IC3 baseline**
 - No
 - It follows the current ISR acknowledgment rule with a separate minimum-ISR admission setting.
 - **IC4 stretch**
 - Contrast broker replication with a consensus metadata controller instead of conflating them.
- Q2: Producer timed out after a successful append?
 - **IC3 baseline**
 - Retry the same supported producer identity and sequence.
 - **IC4 stretch**
 - Use a business event ID for duplicate intent across producer sessions.
- Q3: Consumer wrote the database and crashed before offset commit?
 - **IC3 baseline**
 - It replays, so deduplicate with the database effect in one transaction.
 - **IC4 stretch**
 - Use an outbox or transactional destination pattern for additional emitted events.
- Q4: Can parallel processing commit the highest completed offset?
 - **IC3 baseline**
 - Only if every lower offset in the assigned range completed too.
 - **IC4 stretch**
 - Track contiguous progress and bound in-flight gaps to control replay cost.
- Q5 (premise reversal): One global order is required?
 - **IC3 baseline**
 - One partition creates a global serialization bottleneck.
 - **IC4 stretch**
 - Negotiate whether only related keys need order before paying for a sequencer and its availability limit.
- Q6: Can adding partitions preserve order automatically?
 - **IC3 baseline**
 - No
 - A key can move while old records remain in its former partition.
 - **IC4 stretch**
 - Use a routing-version migration with producer fencing and a drain boundary.

- Q7 (premise reversal): Skip the log and have producers call consumer APIs directly; why pay for retention?
 - **IC3 baseline**
 - Direct calls couple producer availability to every consumer and lose replay, so a consumer bug or outage drops events.
 - The log decouples them and turns a consumer outage into lag instead of loss.
 - **IC4 stretch**
 - Choose direct calls or a task queue when there is one consumer, no replay need, and an idempotent destination.
 - Keep an outbox at the producer so the transport can change later without losing events.
- Validation scenarios
 - Kill a leader before and after acknowledgment and verify the stated failure contract.
 - Retry timed-out producer sequences and verify one accepted log append.
 - Drop ISR below the configured minimum and reject writes.
 - Complete consumer offsets out of order and verify only contiguous progress is committed.
 - Crash after destination commit and replay without a second effect.
 - Increase partition count under keyed traffic using the declared routing migration.

Recall card

- "Ordering is per partition."
- "Replication acknowledgments need a stated failure model."
- "An offset is progress, not proof of an external effect."
- "Commit the contiguous completed prefix."
- Wrap-up script
 - "The log appends ordered records to replicated partitions and exposes a declared visibility boundary."
 - "Producer identities handle retry ambiguity within the supported session contract."
 - "Consumers own destination idempotency and commit only contiguous completed progress."
 - "My weakest assumption is that the storage contract matches the stated durability; retention, hot partitions, and correlated storage failures define the practical limits."

References

- [Ranked bank](#): prompt and provenance.
- [Role framing](#): IC3 baseline, IC4 stretch, and round clock.
- [OCI role framing](#): role decode and scoring context.
- [API/schema companion](#): related contract drills.
- Reference check: 2026-09-10.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [Apache Kafka design](#): replication, delivery semantics, and consumer processing boundaries.
- [Kafka producer configuration](#): acknowledgments and idempotent producer requirements.
- [Kafka broker configuration](#): minimum ISR and election settings.